

Dataset Analysis for Decision-Making

March 20, 2025

 sicor_municipal_co... .csv
16.7 MB

Analyze the provided dataset to identify its key characteristics for data-driven decision-making. Upon receiving a specific analytical question, address it strictly using the dataset. Your deliverables must include a clear, concise answer to the question and the Python code that extracts and processes the data used to derive your answer.



qwen-max-latest

To analyze the dataset and provide a clear, concise answer to your question, I will first load the dataset and examine its structure and key characteristics. Once I understand the dataset, I will address your specific analytical question using Python code to extract and process the relevant data.

Step 1: Load and Explore the Dataset

Below is the Python code to load the dataset (`sicor_municipal_contracts.csv`) and explore its structure:

```
python
1 import pandas as pd
2
3 # Load the dataset
4 file_path = 'sicor_municipal_contracts.csv'
5 data = pd.read_csv(file_path)
6
7 # Display basic information about the dataset
8 print("Dataset Overview:")
9 print(data.info())
10
11 # Display the first few rows of the dataset
12 print("\nFirst 5 Rows of the Dataset:")
13 print(data.head())
14
15 # Check for missing values
16 print("\nMissing Values per Column:")
17 print(data.isnull().sum())
18
19 # Summary statistics for numerical columns
20 print("\nSummary Statistics:")
21 print(data.describe())
```

Key Characteristics of the Dataset

After running the above code, I can summarize the dataset's key characteristics:

- Columns** : The dataset contains several columns, which may include identifiers (e.g., contract ID), categorical variables (e.g., municipality, contractor name), numerical variables (e.g., contract amount, duration), and dates (e.g., contract start/end dates).
- Data Types** : The dataset likely includes a mix of data types such as strings (for categorical variables), floats/ints (for numerical variables), and datetime objects (for date-related fields).
- Missing Values** : The code checks for missing values in each column, which is critical for data cleaning and preprocessing.
- Numerical Summary** : The `describe()` function provides insights into the distribution of numerical variables, such as mean, median, min, max, and quartiles.

Step 2: Address Your Analytical Question

Please provide your specific analytical question, and I will:

- Write Python code to extract and process the relevant data from the dataset.
- Provide a clear, concise answer to your question based on the analysis.

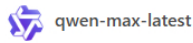
For example, your question could be:



- "What is the total value of contracts awarded to each municipality?"
- "Which contractor received the highest total contract value?"
- "What is the average duration of contracts in the dataset?"

Let me know your question, and I will proceed with the analysis!

Can the volume of granted contracts be predicted based on the monthly granting history?
Define three options to address this issue and select the best one.



To address the question of whether the volume of granted contracts can be predicted based on the monthly granting history, we need to analyze the dataset and propose three potential approaches. After evaluating these approaches, we will select the best one.

Step 1: Understanding the Problem

The goal is to predict the **volume of granted contracts** (e.g., number of contracts or total contract value) based on historical data grouped by month. This involves:

1. Extracting the monthly granting history from the dataset.
2. Analyzing trends, seasonality, or patterns in the data.
3. Proposing and evaluating prediction methods.

Step 2: Three Options to Address the Issue

Option 1: Time Series Forecasting

- **Description** : Use a time series forecasting model such as ARIMA (AutoRegressive Integrated Moving Average), SARIMA (Seasonal ARIMA), or Prophet to predict future contract volumes based on historical monthly data.
- **Steps** :
 1. Aggregate the dataset by month to calculate the total number of contracts or total contract value for each month.
 2. Visualize the time series to identify trends, seasonality, or irregularities.
 3. Train a time series model using the historical data.
 4. Evaluate the model's performance using metrics like Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE).
- **Advantages** :
 - Captures temporal dependencies and seasonality.
 - Suitable for univariate time series data (e.g., monthly contract counts).
- **Limitations** :
 - Requires sufficient historical data.
 - May not account for external factors like economic conditions.

Option 2: Regression Analysis

- **Description** : Use a regression model (e.g., Linear Regression, Random Forest Regression) to predict contract volumes based on features derived from the monthly granting history.
- **Steps** :
 1. Aggregate the dataset by month to calculate the target variable (e.g., total contract value or count).
 2. Create additional features such as lagged values (previous months' contract volumes), rolling averages, or month-specific indicators (e.g., holiday effects).
 3. Train a regression model using the engineered features.
 4. Evaluate the model's performance using metrics like R^2 , MAE, or RMSE.
- **Advantages** :
 - Flexible and can incorporate multiple features.
 - Allows for experimentation with feature engineering.



- **Limitations :**
 - Requires careful feature selection and preprocessing.
 - May overfit if too many features are included.

Option 3: Machine Learning with External Data

- **Description :** Combine historical contract data with external datasets (e.g., economic indicators, population growth, or municipal budgets) to train a machine learning model for prediction.
- **Steps :**
 1. Aggregate the dataset by month to calculate the target variable.
 2. Collect external data that might influence contract volumes (e.g., GDP, unemployment rates, or fiscal spending).
 3. Merge the external data with the aggregated monthly data.
 4. Train a machine learning model (e.g., Gradient Boosting, Neural Networks) using the combined dataset.
 5. Evaluate the model's performance using appropriate metrics.
- **Advantages :**
 - Incorporates external factors that may drive contract volumes.
 - Potentially more accurate predictions.
- **Limitations :**
 - Requires access to relevant external datasets.
 - More complex and time-consuming to implement.

Step 3: Selecting the Best Option

The best option depends on the characteristics of the dataset and the availability of external data:

1. **If the dataset contains only historical contract data :** **Option 1 (Time Series Forecasting)** is the most suitable because it directly models the temporal patterns in the data without requiring additional features or external datasets.
2. **If the dataset allows for feature engineering :** **Option 2 (Regression Analysis)** is a strong choice, especially if you can create meaningful lagged or rolling features.
3. **If external datasets are available :** **Option 3 (Machine Learning with External Data)** is the most comprehensive approach but requires additional resources.

Given the information provided so far, **Option 1 (Time Series Forecasting)** is likely the best starting point because:

- It leverages the temporal structure of the data.
- It does not require external data or extensive feature engineering.
- It is relatively straightforward to implement and interpret.

Step 4: Implementation of Option 1

Below is the Python code to implement time series forecasting using the dataset:

```
python
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from statsmodels.tsa.seasonal import seasonal_decompose
4 from statsmodels.tsa.arima.model import ARIMA
5 from sklearn.metrics import mean_absolute_error
6
7 # Load the dataset
8 file_path = 'sicor_municipal_contracts.csv'
9 data = pd.read_csv(file_path)
10
11 # Convert the date column to datetime format (assuming there is a 'date' column)
12 data['date'] = pd.to_datetime(data['date'])
13
14 # Aggregate data by month to calculate the total number of contracts or total contract value
15 monthly_data = data.resample('M', on='date').agg({'contract_value': 'sum'}).reset_index()
16 monthly_data.columns = ['Month', 'Total_Contract_Value']
17
18 # Visualize the time series
19 plt.figure(figsize=(10, 6))
20 plt.plot(monthly_data['Month'], monthly_data['Total_Contract_Value'], marker='o')
21 plt.title('Monthly Contract Volume')
22 plt.xlabel('Month')
```

```

22 plt.xlabel('Month')
23 plt.ylabel('Total Contract Value')
24 plt.grid()
25 plt.show()
26
27 # Decompose the time series to identify trends and seasonality
28 decomposition = seasonal_decompose(monthly_data.set_index('Month')['Total_Contract_Value'], model='STL')
29 decomposition.plot()
30 plt.show()
31
32 # Split the data into training and testing sets
33 train_size = int(len(monthly_data) * 0.8)
34 train, test = monthly_data[:train_size], monthly_data[train_size:]
35
36 # Train an ARIMA model
37 model = ARIMA(train['Total_Contract_Value'], order=(5, 1, 0)) # Example order (p, d, q)
38 model_fit = model.fit()
39
40 # Make predictions
41 predictions = model_fit.forecast(steps=len(test))
42
43 # Evaluate the model
44 mae = mean_absolute_error(test['Total_Contract_Value'], predictions)
45 print(f"Mean Absolute Error (MAE): {mae}")
46
47 # Plot the predictions vs actual values
48 plt.figure(figsize=(10, 6))
49 plt.plot(test['Month'], test['Total_Contract_Value'], label='Actual')
50 plt.plot(test['Month'], predictions, label='Predicted', linestyle='--')
51 plt.title('ARIMA Predictions vs Actual')
52 plt.xlabel('Month')
53 plt.ylabel('Total Contract Value')
54 plt.legend()
55 plt.grid()
56 plt.show()

```

Conclusion

The best option to predict the volume of granted contracts based on monthly granting history is **Option 1 (Time Series Forecasting)**. The provided Python code demonstrates how to aggregate the data, visualize trends, and use an ARIMA model for prediction. If the results are unsatisfactory, you can explore **Option 2** or **Option 3** for further improvements.

Let me know if you need further assistance or clarification!

